

Python Walrus Operator (:=)

The **walrus operator (:=)** is a new type of assignment operator that was introduced in Python 3.8. This chapter will give a clear understanding of the walrus operator and how to use it to reduce number of lines in your Python code.

What is Walrus Operator?

The walrus operator (:=) is used to assign a value to a variable from an expression inside loop conditions. This is useful when you need to use a value multiple times in a loop, but don't want to repeat the calculation. Hence it is also known as the **assignment expression operator**. The name "walrus operator" comes because the operator (:=) looks like the eyes and tusks of a walrus.



The **syntax** of the walrus operator is as follows –

```
variable := expression
```

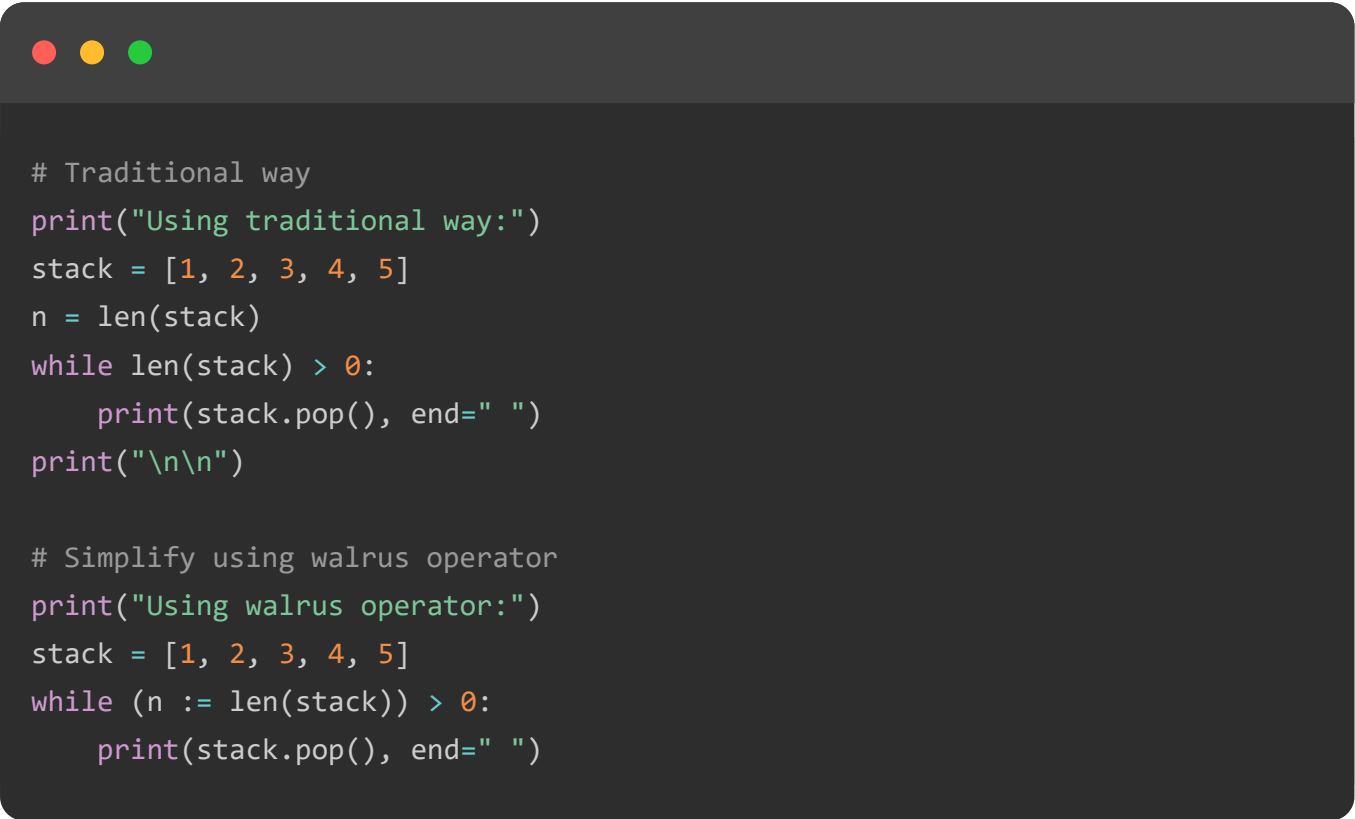
```
# Example
n := len(arr) + 10
```

- **variable** – This is the name of the variable to which the value will be assigned.
- **expression** – The mathematical expression, function call, or any valid Python expression that returns a value.

Example of Walrus Operator

Now, let's look at an example to understand how the walrus operator can be used to simplify code. Consider the following code that uses a **while loop** to pop elements from a stack until it is empty. At each iteration of the loop, we calculate the length of the stack using **len(stack)** and assign it to the variable **n**.

The walrus operator allows us to combine these two operations into a single line, see the code below –



```
# Traditional way
print("Using traditional way:")
stack = [1, 2, 3, 4, 5]
n = len(stack)
while len(stack) > 0:
    print(stack.pop(), end=" ")
print("\n\n")

# Simplify using walrus operator
print("Using walrus operator:")
stack = [1, 2, 3, 4, 5]
while (n := len(stack)) > 0:
    print(stack.pop(), end=" ")
```

The **output** of both the above code will be –

```
Using traditional way:
5 4 3 2 1
```

Using walrus operator:

5 4 3 2 1

Walrus Operator for List Comprehensions

The **list comprehensions** is a simple way to create lists in Python using a single line of code. The walrus operator can be used in list comprehensions to assign a value to a variable as part of the expression.

In the **example** below, we show how list comprehensions and the walrus operator simplify the code that generates a list of squares of numbers greater than 20 from a range of 10 numbers.

```
# Traditional way
print("Using traditional way:")
results = []
for x in range(10):
    y = x * x
    if y > 20:
        results.append(y)
print(results, "\n")

# Simplify using walrus operator
print("Using walrus operator:")
results = [y for x in range(10) if (y := x * x) > 20]
print(results)
```

The **output** of both the above code will be –

Using traditional way:

[25, 36, 49, 64, 81]

Using walrus operator:

[25, 36, 49, 64, 81]

When to Avoid Using Walrus Operator?

We already saw that the walrus operator can be used to reduce the number of lines in your code and make it more readable. However, there are some scenarios where using the walrus operator may not be appropriate.

Avoid using the walrus operator in following scenarios –

- The assignment expression is too complex and makes the code harder to read.
- Direct assignment to a variable is restricted. For example, **`x := 10`** will result in a syntax error.
- The walrus operator is not allowed in lambda functions. For example, **`lambda x := 5: x * 2`** will result in a syntax error.

Conclusion

If you are using Python 3.8 or later, you can use the walrus operator to reduce the number of lines in your code and make it more readable. However, it is important to use the walrus operator carefully, because it can make code harder to read if overused or used inappropriately. It is recommended to use the walrus operator only when it improves the clarity of the code.

TOP TUTORIALS

[Python Tutorial](#)

[Java Tutorial](#)

[C++ Tutorial](#)

[C Programming Tutorial](#)

[C# Tutorial](#)

[PHP Tutorial](#)

[R Tutorial](#)

[HTML Tutorial](#)

[CSS Tutorial](#)

[JavaScript Tutorial](#)

[SQL Tutorial](#)

TRENDING TECHNOLOGIES

[Cloud Computing Tutorial](#)

[Amazon Web Services Tutorial](#)

Microsoft Azure Tutorial
Git Tutorial
Ethical Hacking Tutorial
Docker Tutorial
Kubernetes Tutorial
DSA Tutorial
Spring Boot Tutorial
SDLC Tutorial
Unix Tutorial

CERTIFICATIONS



Chapters ▾

Categories ≡

Data Science Advanced Certification
Cloud Computing And DevOps
Advanced Certification In Business Analytics
Artificial Intelligence And Machine Learning
DevOps Certification
Game Development Certification
Front-End Developer Certification
AWS Certification Training
Python Programming Certification

COMPILERS & EDITORS

Online Java Compiler
Online Python Compiler
Online Go Compiler
Online C Compiler
Online C++ Compiler
Online C# Compiler
Online PHP Compiler
Online MATLAB Compiler
Online Bash Terminal
Online SQL Compiler
Online Html Editor

[ABOUT US](#) | [OUR TEAM](#) | [CAREERS](#) | [JOBS](#) | [CONTACT US](#) | [TERMS OF USE](#) |
[PRIVACY POLICY](#) | [REFUND POLICY](#) | [COOKIES POLICY](#) | [FAQ'S](#)



Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.

© Copyright 2025. All Rights Reserved.